

Lab 13 — Fine-Tuning a Domain-Specific Academic Assistant (LLMs)

Professional report format (IEEE-inspired). Evidence-backed evaluation: base vs LoRA fine-tuned `distilgpt2`, failure analysis, and one manual dataset improvement cycle.

STUDENT & SETUP

Student

Zarmeena Jawad

Roll No

B23F0115AI125

Course

COMP-341L

Section

B.S AI - Red

Lab

13

Date

May 05, 2026

Base Model

`distilgpt2`

Fine-tuning

LoRA (PEFT)

Manual dataset (JSONL)

Style template enforced

Base vs tuned comparison

Failure-driven iteration

The goal of this lab is not just code execution: it prioritizes reasoning, analysis, and decision-making about model behavior, limitations, and improvements.

ARTIFACTS

Notebook: `lab13_llm_finetune_academic_assistant_colab.ipynb`

Dataset v1: `data/dataset_cs_instructions_v1.jsonl`

Dataset v2: `data/dataset_cs_instructions_v2.jsonl`

Evidence: `eval_*.txt`, `failures_*.json`

IEEE-STYLE REPORT

ABSTRACT

This work fine-tunes a small language model to behave as an academic assistant that explains Computer Science concepts in a strict teaching style. A curated instruction dataset is written manually and used to train LoRA adapters on `distilgpt2`. The system is evaluated by comparing base vs fine-tuned outputs on the same prompt, and analyzed through targeted failure cases. A manual improvement cycle is attempted via dataset refinement.

1. OBJECTIVE & CONSTRAINTS

- Improve **clarity**, **style consistency**, and reduce **overconfident guessing**.
- Use a **small model** that can be fine-tuned under Colab constraints.
- Do not submit unmodified AI text as “dataset explanations”; explanations are written manually.

2. TEACHING STYLE DEFINITION

Template enforced in every dataset response:

- Step-by-step explanation
- Real-life analogy
- Tiny example (1–3 lines)
- Common mistake

- One-line recap

3. DATASET (MANUAL)

The dataset is stored as JSONL with fields `instruction` and `output`. Each output is written to follow the defined teaching style.

Dataset	Samples	Purpose
v1	25+	Primary style tuning across CS topics
v2	5	Failure-driven improvement (DP state selection + uncertainty handling)

4. MODEL SELECTION

Model: distilgpt2. Selected for fast iteration and low compute cost. The primary limitation is reduced factual reliability and higher repetition risk compared to larger LLMs.

5. FINE-TUNING STRATEGY (LORA)

LoRA trains a small number of additional parameters (adapters) while freezing the base model. This reduces memory usage and makes fine-tuning feasible on Colab GPUs while enabling direct comparison between base and adapted behavior.

6. EVALUATION (SAME PROMPT: “EXPLAIN DYNAMIC PROGRAMMING”)

Evidence is presented as raw model outputs saved in `eval_*.txt`.

6.1 Base Model Output

```
### Instruction:
Explain dynamic programming in a step-by-step, analogy-based teaching style.

### Response:
This is a great introduction to the following topics, which are covered in this tutorial.
Note: This is not a complete tutorial.
### The lesson is a part of the course, so please read the full course description.
```

6.2 Fine-Tuned Output (v1)

```
### Instruction:
Explain dynamic programming in a step-by-step, analogy-based teaching style.

### Response:
This post is part of a series that focuses on the language of programming in a way that is not currently possible in the language of programming. It has been translated into English and Spanish.
The most recent version of this article is available on the Web.
```

6.3 Fine-Tuned Output (v2)

[illegible]

going to say that I am going to say that I am going to say that I am going to say that I am going

6.4 Analysis

- **Clarity:** The base response is generic and does not explain DP. Fine-tuned outputs are off-topic or repetitive.
- **Hallucination/overconfidence:** The tuned model does not reliably adopt a cautious stance; it generates “article-like” filler without grounded explanation.
- **Style consistency:** The strict template (steps, analogy, example, mistake, recap) is not consistently produced, implying insufficient data/epochs/decoding stability for robust style control.

7. FAILURE ANALYSIS (2 CASES)

Two prompts were used to expose errors: (i) proof-like requests and (ii) requests for precise external facts.

7.1 Proof request failure

Asking for a proof (e.g., “prove DP always gives optimal answer”) fails because the dataset does not teach formal proof framing (optimal substructure assumptions, state definition, recurrence justification). This is a **data coverage** gap plus a **small-model limitation**.

7.2 External-fact precision failure

Requests like “best-known TSP algorithm complexity and discovery year” encourage hallucination because the model lacks reliable access to up-to-date citations. Without uncertainty examples, it may guess. This is primarily a **model limitation** and **training-data policy gap** (missing “I don’t know” patterns).

8. MANUAL IMPROVEMENT (ONE FIX)

The v2 dataset adds examples that explicitly teach (a) DP state selection and (b) hallucination awareness / safer behavior. However, since v2 is only a few samples, it is not sufficient to reliably override unstable generation behavior.

9. KEY INSIGHTS

- With tiny datasets, fine-tuning can amplify repetition and irrelevant patterns instead of enforcing structure.
- Style control requires **more high-signal examples**, especially for the evaluation concept (DP here).
- Safer academic behavior benefits from explicit “uncertainty/ask clarifying question” training examples.

REFERENCES

1. Hugging Face Transformers documentation.
2. PEFT (Parameter-Efficient Fine-Tuning) documentation.
3. LoRA: Low-Rank Adaptation for efficient fine-tuning.
4. DistilGPT-2 / GPT-2 model documentation and model cards.

Evidence files: `eval_base.txt`, `eval_finetuned_v1.txt`, `eval_finetuned_v2.txt`, `failures_*.json`.